CSE398/498 Generative AI for Healthcare & Robotics

Lab 2 Report — Redo

ReVA video question answering: Qwen evaluation, LoRA fine-tuning, and VILA

Xiyuan Xu | Redone Oct 12–13, 2026 | Instructor: Professor Chuah

GitHub: https://github.com/LeoXu227/VLM_evaluating_finetuing

What I implemented

This lab has three student-code parts. First, I finished the ReVA-to-Qwen training-data converter in `scripts/build_qwen_train_data.py`. It walks through the nested ReVA annotations, formats the multiple-choice options, adds the video path, and creates the conversation format Qwen expects.

Second, I finished the ReVA test-set preparation and scoring code. The preparation script flattens the original annotations into one row per question with a stable ID. The scorer extracts the answer letter from the model response. A missing or bad answer counts as wrong instead of disappearing from the denominator.

Third, I finished the VILA adapter in `vila_eval/reva_v2.py`: loading questions, finding each video, building the prompt, parsing VILA's answer, and summarizing accuracy. These are the parts marked `TODO(student)` in the starter.

1. What this lab is testing

ReVA is a video question-answering benchmark made from aerial and remote-sensing videos. A question might ask how the scene changes, where one object is compared with another, how long something stays visible, or what will probably happen next. Every question has answer choices, so the output can be scored as one letter.

The main comparison is: (1) Qwen3-VL before training, (2) the same Qwen model after LoRA fine-tuning on ReVA examples, and (3) VILA1.5-3B as a second vision-language model. I expected fine-tuning to help Qwen because the training examples use the same kind of aerial-video questions as the test set.

My first report only had five smoke-test questions. That was enough to prove the code ran, but not enough to support a result. A later 110-question subset still over-weighted rare types. This redo reports the official 4,000-question test set.

1.1 How one question goes through the pipeline

For each test item, the preparation script finds the video and reads its duration. The evaluator samples up to four frames, then sends the frames, question, and answer choices to the model. Qwen is told to put its final choice inside `<answer>...</answer>`. VILA is told to return only the option letter.

The scorer compares that letter with the ReVA ground truth. I report two numbers: **completion** (did the model return a parseable answer?) and **accuracy** (how many of all selected questions were correct). This matters because a model should not get a better score just because failed outputs were skipped.

2. Experiment setting

I ran the real ReVA data on the MAGIC server. The dataset at `/data/datasets/ReVA` is about 28 GB, and the provided test annotations flatten to 4,000 questions. The models were `Qwen/Qwen3-VL-4B-Instruct` and `Efficient-Large-Model/VILA1.5-3b`.

The MAGIC copy of ReVA flattens to **4,000 test questions from 1,014 videos**. I used that full list, not a convenience sample. The 11 ReVA types keep their original counts (for example 660 object/land-cover questions and 100 consequence questions). The same 4,000 IDs were scored for Qwen base, Qwen LoRA, and VILA.

I did run a 110-question balanced subset (seed 398, ten per type) first. On that subset LoRA looked slightly worse than base (70.00% vs 72.73%). That gap was three questions. The full test was the check on whether that drop was real.

Setting	Value
Server / GPU	MAGIC magic01; RTX 2080 Ti 11 GB
Evaluation set	4,000 questions; official ReVA test split
Videos represented	1,014
Video input	Up to 4 frames; max 50,176 pixels
Qwen inference	Transformers backend; 3 shards on GPU 1, 2, 3; batch 1
VILA inference	GPU 0 in parallel; 4 video frames
Missing / unparsed answers	Counted as wrong

2.1 Fine-tuning setting

I trained a new LoRA checkpoint for this report so the setup would be fully logged. Training used 200 ReVA examples for one epoch. I loaded the Qwen base model in 4-bit mode to fit the 2080 Ti and trained LoRA adapters on the Q, K, V, and output projection layers. Rank was 8, alpha 16, dropout 0.05, learning rate 2e-7, batch size 1, gradient accumulation 8, FP16, and maximum sequence length 1,024. Training finished 25 optimizer steps in about 175 seconds.

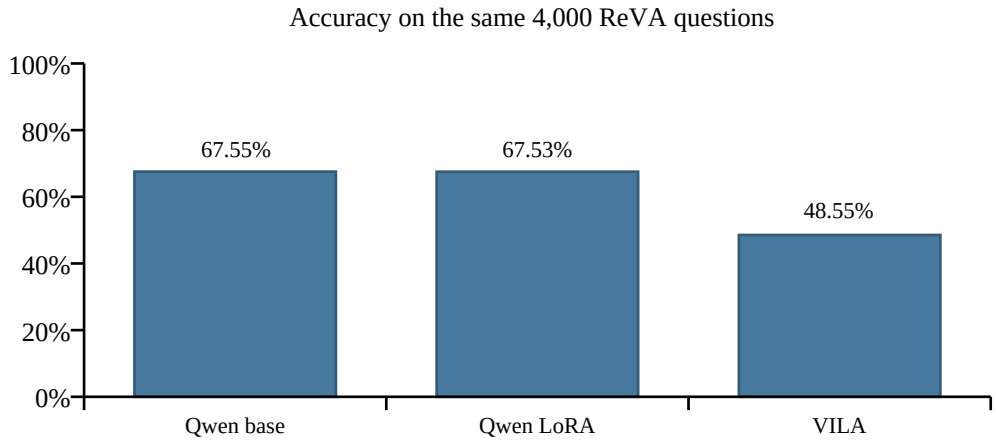
3. What I ran on MAGIC

I first ran base Qwen on GPUs 1–3 (GPU 0 had another user's leftover kernels, and an earlier attempt there ran out of memory). Then I ran the LoRA checkpoint with the same frames, pixels, prompt, and scorer. VILA ran on GPU 0 at the same time as LoRA so it would not add another hour after Qwen finished. After scoring, I checked that all three files contain the same 4,000 IDs.

VILA hit one hardware issue. Its SigLIP vision code forced FlashAttention 2, but FlashAttention 2 requires an Ampere-or-newer GPU and MAGIC has Turing 2080 Ti cards. I added a fallback to the normal PyTorch/SigLIP attention path for older GPUs. This does the same attention operation more slowly; it does not change the model weights. After that, VILA completed all 4,000 questions.

3.1 Overall results

Model	Completed	Correct	Accuracy
Qwen3-VL base	3984/4000	2702/4000	67.55%
Qwen3-VL LoRA	3977/4000	2701/4000	67.53%
VILA1.5-3B	4000/4000	1942/4000	48.55%



On the full test, base Qwen and LoRA are a **tie**: 67.55% vs 67.53%. The Wilson 95% intervals (66.08–68.98% vs 66.06–68.96%) sit on top of each other. That is the result the 110-question subset could not support. With 200 training examples and one epoch, LoRA moved many individual answers but did not change overall accuracy.

In the paired comparison, base-only-correct was 99 questions and LoRA-only-correct was 98. McNemar's test with continuity correction gave p=1.0, so I cannot claim either Qwen model is better. Qwen base versus VILA is a real gap: 1,132 base-only correct vs 372 VILA-only ($p \approx 10^{-85}$). VILA's 48.55% is well below both Qwen models.

3.2 Results by question type

Question type (n)	Base	LoRA	VILA
Causation Reasoning (180)	165/180	164/180	135/180
Change Detection (500)	307/500	310/500	229/500
Consequence Reasoning (100)	88/100	89/100	77/100
General Understanding (180)	159/180	158/180	152/180
Geometric Relation (400)	275/400	271/400	152/400
Hypothetical Reasoning (160)	138/160	141/160	86/160
Object / Land Cover (660)	472/660	477/660	356/660
Perspective / Viewpoint (480)	286/480	291/480	208/480
Structural Layout (340)	255/340	252/340	179/340
Temporal Grounding (640)	302/640	288/640	196/640
Trend and Pattern (360)	255/360	260/360	172/360

The easiest group for Qwen was **Causation Reasoning** (about 91%). **General Understanding** and **Consequence Reasoning** were also high. The hardest group was still **Temporal Grounding**: base 302/640 (47.2%), LoRA 288/640 (45.0%), VILA 196/640 (30.6%). Four sampled frames can miss when an object enters or leaves, and the model has to reason about the whole clip.

LoRA's type-level changes are small and mixed. It picked up a few object, change, trend, perspective, and hypothetical questions, and lost some temporal and geometric ones. None of that moves the overall score. VILA is behind Qwen on every type, especially geometric relation (152/400 vs 275/400) and temporal grounding.

3.3 Limits of these numbers

These are full-test accuracies, but they are still one training seed, one LoRA recipe (200 examples, one epoch, rank 8), and one four-frame budget. I did not search hyperparameters. Qwen left 16 (base) and 23 (LoRA) answers unparsed; those count as wrong. The first training step logged a NaN gradient norm, but its loss was finite; the next 24 gradient norms and all losses were finite, and the run completed normally. I kept that line in the log instead of hiding it. Qwen and VILA also use different conversation templates, so the IDs and video budget match but the text prompts are not byte-for-byte identical.

4. Three examples from the same test set

4.1 Geometric relation — one LoRA improvement



Figure 1. Three frames from REVA-G003965. The question asks where the parking lot is relative to the road and soccer goal.

Ground truth C. Base B; LoRA C; VILA A. This is one question LoRA fixed while base was wrong. The scene needs a spatial relation across the road, field, and parking lot. It shows that fine-tuning can help a single case, even though the full-test total did not move.

4.2 Change detection — a fine-tuning regression

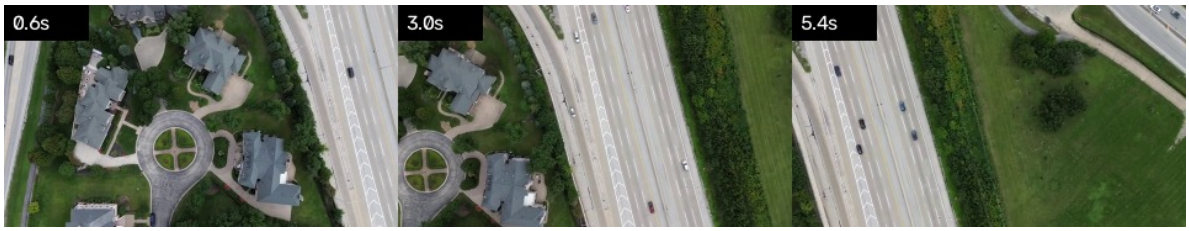


Figure 2. Three frames from REVA-G029758. The aerial view moves from the houses and circular driveway toward the highway.

Ground truth D. Base D; LoRA C; VILA B. Base Qwen got this change right, but the trained checkpoint changed to the wrong answer. The full test has 99 such base-only wins and 98 LoRA-only wins, which is why the totals match.

4.3 Temporal grounding — VILA gets one Qwen misses



Figure 3. Three frames from QA-011303. The pond stays visible across most of this short clip.

Ground truth A. Base C; LoRA C; VILA A. Both Qwen models said the pond was only visible briefly at the beginning. VILA correctly said it was visible for most of the video. This is a temporal reasoning error, not answer parsing: all three models returned valid letters.

5. How to rerun my experiment

The code is in my GitHub repository: https://github.com/LeoXu227/VLM_evaluating_finetuing. The redo package contains the report, scored CSV summaries, verification file, and the scripts used on MAGIC.

Run the full 4,000-question pipeline:

```
bash redo_lab2/scripts/run_full_4000_pipeline.sh
```

Or run base Qwen only:

```
RESUME=1 EVAL_BATCH_SIZE=1 MODEL_PATH=Qwen/Qwen3-VL-4B-Instruct EVAL_NAME=full_qwen_base_n4000 GPU_IDS=1,2,3
MAX_FRAMES=4 MAX_PIXELS=50176 BACKEND=transformers GT_FILE=$PWD/redo_lab2/data/reva_full_4000_flat.json bash
redo_lab2/scripts/run_qwen_eval_2gpu.sh
```

Run trained Qwen:

```
EVAL_BATCH_SIZE=1 MODEL_PATH=$PWD/redo_lab2/outputs/qwen_reva_sft_explicit
MODEL_BASE=Qwen/Qwen3-VL-4B-Instruct EVAL_NAME=full_qwen_lora_n4000 GPU_IDS=1,2,3 MAX_FRAMES=4
MAX_PIXELS=50176 BACKEND=transformers GT_FILE=$PWD/redo_lab2/data/reva_full_4000_flat.json bash
redo_lab2/scripts/run_qwen_eval_2gpu.sh
```

Run VILA:

```
CONDA_ENV=vila VILA_REPO=$HOME/VILA MODEL_PATH=Efficient-Large-Model/VILA1.5-3b REVA_ROOT=/data/datasets/ReVA
REVA_JSON=/data/datasets/ReVA/test_set.json OUTPUT_DIR=$PWD/redo_lab2/outputs/full_vila_n4000
NUM_VIDEO_FRAMES=4 GPU=0 bash scripts/run_eval_reva_vila.sh
```

6. Conclusion

This redo is the comparison I can defend: the same 4,000 ReVA questions, same saved IDs, four frames per video, and every missing answer counted. Qwen base scored 67.55%, LoRA scored 67.53%, and VILA scored 48.55%. Fine-tuning did not help overall, and the 110-question drop was sampling noise. The clearest shared weakness is still temporal grounding.

A better follow-up would repeat LoRA with several seeds and a larger training set. That would show whether a different recipe can move the 4,000-question score, or whether Qwen is already saturating this four-frame setup.

Appendix

AI tools

I used Cursor/AI to help debug the MAGIC environment, write the experiment scripts, organize the rerun, and edit this report. I ran the training/evaluations on my MAGIC account, kept the raw logs and predictions, and can explain the data preparation, LoRA setting, scoring, and model comparison (course AI policy).

References

1. My repository: https://github.com/LeoXu227/VLM_evaluating_finetuing
2. Course Lab 2 assignment and starter repository
3. ReVA dataset: https://huggingface.co/datasets/ReVA-Benchmark/ReVA
4. Qwen3-VL-4B-Instruct: https://huggingface.co/Qwen/Qwen3-VL-4B-Instruct
5. VILA1.5-3B: https://huggingface.co/Efficient-Large-Model/VILA1.5-3b
6. PEFT/LoRA documentation: https://huggingface.co/docs/peft/